Báo cáo kết quả của phương pháp Quantum cho bài báo Representation-Steered Incremental Adapter-Tuning

Tên bài báo gốc: Representation-Steered Incremental Adapter-Tuning for Class-Incremental Learning with Pre-Trained Models

Hướng nghiên cứu Khảo sát thước đo tương đồng lượng tử mô phỏng trên nền phương pháp RSIAT

Nhóm sinh viên thực hiện: Nhóm số 4

***Hà Nội*, ngày 05 tháng 11 năm 2026**

Bài toán và mục tiêu nghiên cứu

Bài toán: Class Increamental Learning (CIL) theo thiết lập exemplar-free.

- Dữ liệu được chia thành nhiều task. Ở mỗi task, mô hình chỉ nhận ảnh của các lớp mới, nhưng khi kiểm tra, mô hình phải phân loại trên toàn bộ các lớp đã học từ đầu đến hiện tại và không được biết ảnh thuộc task nào.

Khó khăn: Catastrophic Forgetting

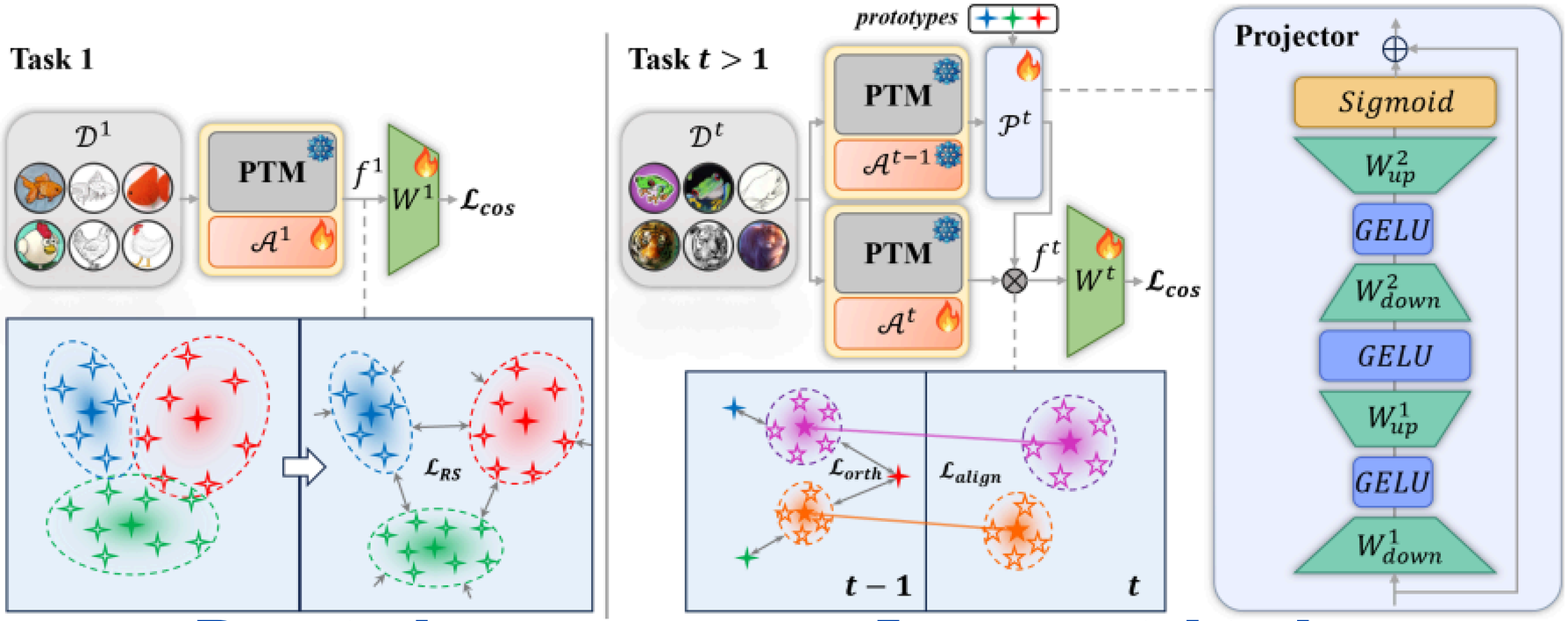
- Khi adapter được cập nhật để học lớp mới, không gian đặc trưng có thể dịch chuyển. Những đặc trưng và ranh giới quyết định từng phù hợp với lớp cũ vì thế có thể không còn phù hợp nữa. Trong khi đó, thiết lập exemplar-free không cho phép giữ lại ảnh cũ để huấn luyện lại trực tiếp.

Câu hỏi nghiên cứu:

- Liệu một projected local quantum kernel có tạo ra tín hiệu representation steering hữu ích hơn trong quá trình học tuần tự hay không? (QKRS)

Sử dụng nền tảng của RSIAT

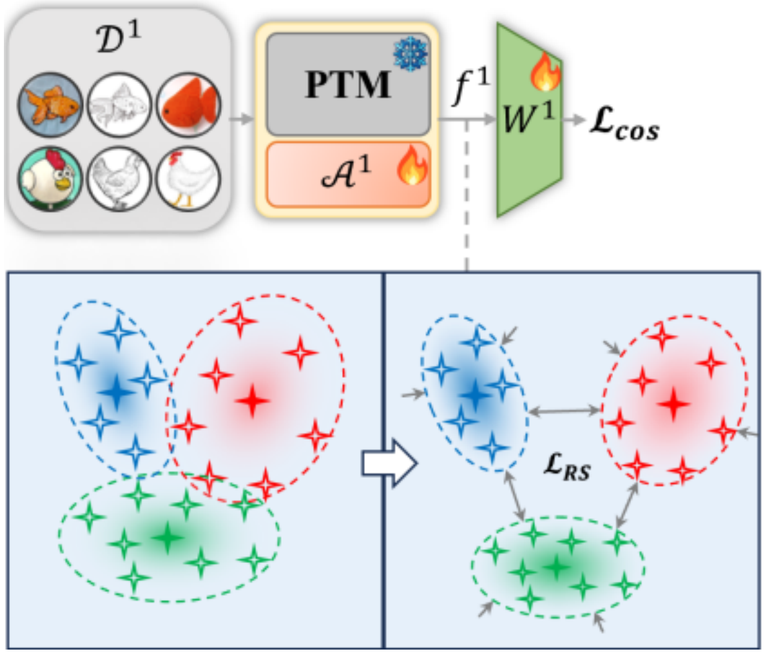
Phương pháp trong bài báo này sử dụng **ViT pretrained**, đóng băng phần **backbone** chính và sử dụng một bộ **adapter chung** qua các task.



Base task **Incremental task**

Sử dụng nền tảng của RSIAT

Base task



Luồng hoạt động:

Ảnh đi qua ViT pretrained và shared adapter để tạo feature

↓

Các feature đã được học sẽ đi qua classifer (W1) để phân biệt

↓

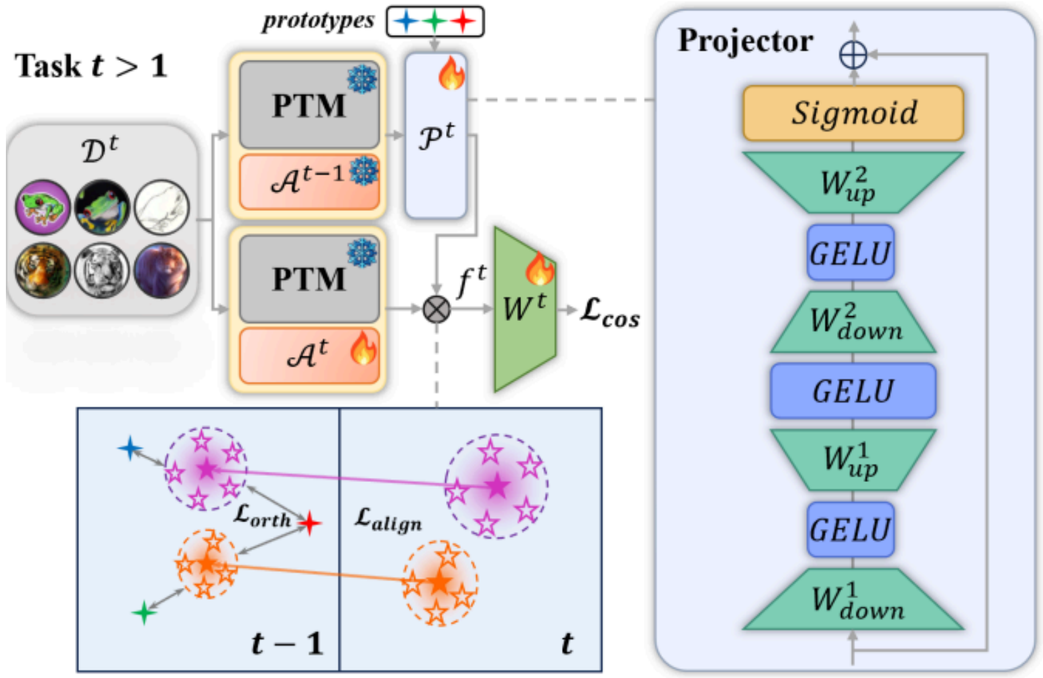
Các feature cũng được tổ chức lại hình học

↓

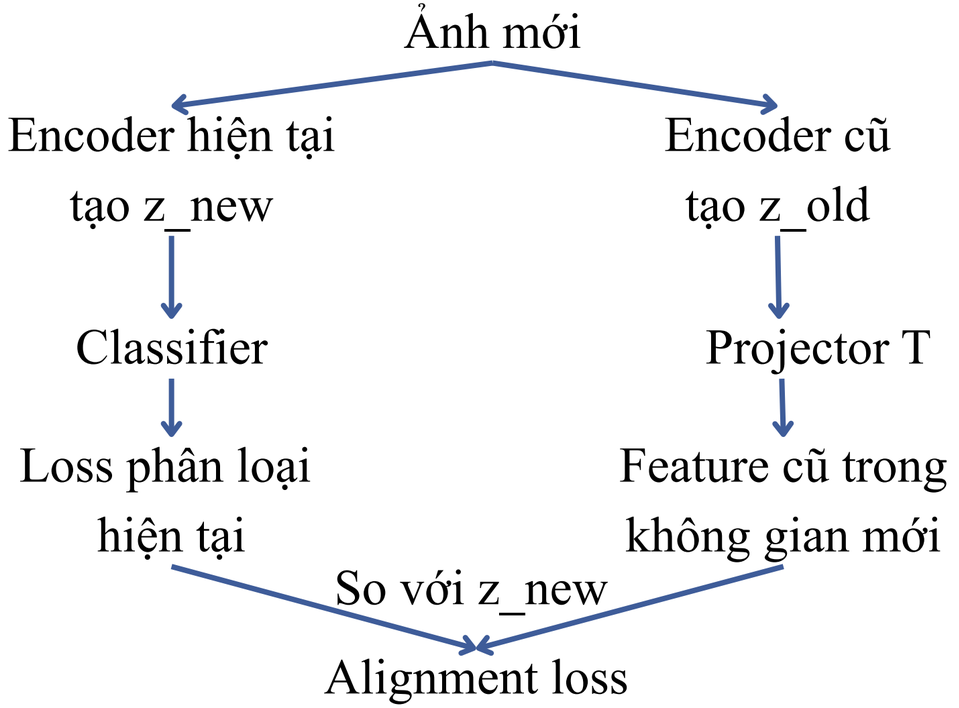
$$\mathcal{L}^{(1)} = \mathcal{L}_{\cos}^{(1)} + \lambda_{\mathrm{RS}}(e)\mathcal{L}_{\mathrm{RS}}$$

Sử dụng nền tảng của RSIAT

Incremental task



Luồng hoạt động:



RSIAT không chỉ dùng classification loss. Nó tác động vào representation trong khi train và sửa classifier sau task.

Contribution: Quantum Kernel Steering Representation

Ý tưởng cốt lõi: Thay vì sử dụng cosine similarity để điều chỉnh hình học của feature space như RSIAT, QKSR thử sử dụng một metric phi tuyến được xây dựng từ quantum circuit mô phỏng.

Mục tiêu là giúp mô hình:

- Học được representation phân biệt tốt các lớp mới.
- Hạn chế sự chồng lấn giữa representation của lớp cũ và lớp mới.
- Giảm catastrophic forgetting khi liên tục bổ sung các lớp mà không lưu ảnh cũ.

Điểm quan trọng là QKSR không thay thế ViT, adapter hay classifier. Nó đóng vai trò regularization trong huấn luyện, tác động đến representation thông qua các hàm loss.

Những thay đổi trong Base Task

1. Ảnh đầu vào qua ViT được các đặc trưng $z_i = f_{\phi_0}(x_i) \in \mathbb{R}^{768}$

2.Sau đó đặc trưng này đi qua layernorm: $\bar{z}_j = \frac{z_j - \mu_z}{\sqrt{v_z + \epsilon}} \in \mathbb{R}^{768}$

(Giảm sự phụ thuộc vào độ lớn và độ lệch trung bình của feature)

3.Sau đó đặc trưng này đi qua layernorm để học các đặc trưng quan trọng: $h = W\bar{z} + b$

Trong contribution hiện tại dùng qubit = 8, mỗi h_k là một tổ hợp tuyến tính được học từ 768 feature dimensions

$$h = [h_1, h_2, \ldots, h_8]$$

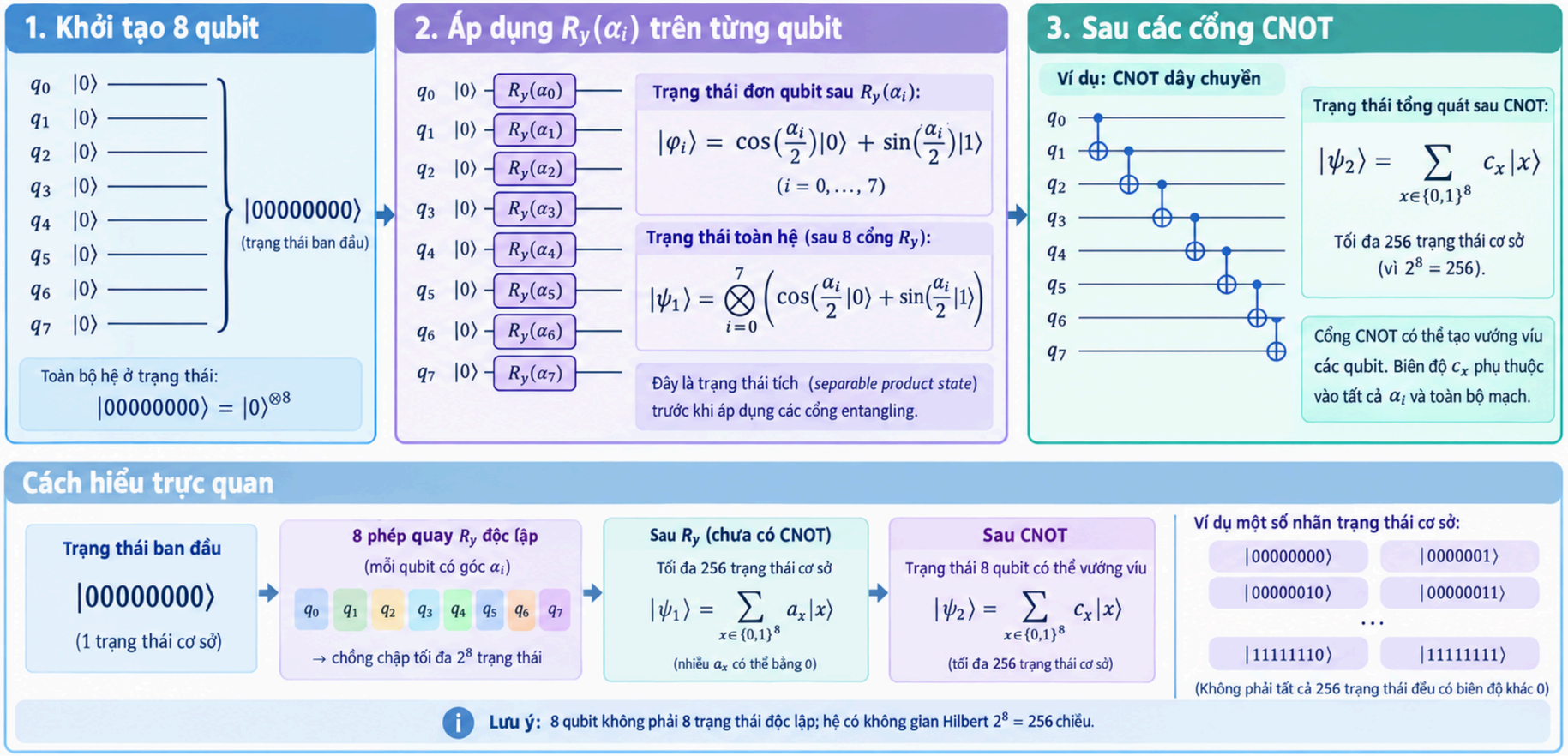
4.Các tổ hợp h_k sẽ đưa về các góc thông qua tanh(.): $a_k = \pi \tanh(h_k)$

Cả phương quá trình trên được tổng hợp theo 1 công thức sau:

$$a_k = \pi \tanh(W\,\mathrm{LN}(z_k) + b)$$

Những thay đổi trong Base Task

Đây là nơi vector góc được chuyển thành một quantum representation, sau khi có 8 góc quay thì chúng sẽ đi qua ma trận cổng R_y và CNOT như sau:



Thu được các pure statevector $|\psi\rangle$

Những thay đổi trong Base Task

Chuyển các statevector thu được thành biểu diễn đặc trưng cục bộ thông qua RDM với p là ma trận mật độ (dày) để được các biểu diễn đặc trưng cục bộ sau:

$$\rho = |\psi\rangle\langle\psi| \quad \Longrightarrow \quad \rho_k = \mathrm{Tr}_{\bar{k}}(\rho)$$

Tính độ lệch/ khoảng cách trực tiếp giữa hai biểu diễn đặc trưng lượng tử thông qua Frobenius Distance

$$D(x_i, x_j) = \sum_{k=0}^{q-1} \|\rho_k(x_i) - \rho_k(x_j)\|_F^2$$

Chuẩn hóa khoảng cách thành điểm số tương đồng để đưa vào Steering Loss thông qua RBF Kernel -Tính độ tương đồng.

$$K(x_i, x_j) = \exp[-\gamma_Q D(x_i, x_j)]$$

QKSR dùng nhãn y_i để điều chỉnh Quantum Kernel Matrix K: Kéo các mẫu cùng lớp lại gần, đẩy các mẫu khác lớp ra xa.

Incremental task: QKSR separation giữa lớp cũ và lớp mới

QKSR tận dụng Quantum Kernel từ Base Task làm công cụ chống quên (Anti-Forgetting) mà không cần lưu trữ dữ liệu cũ.

Ngăn chặn việc học dữ liệu mới ở task hiện tại t làm biến dạng không gian biểu diễn (Representation Space) đã tối ưu ở task trước t-1.

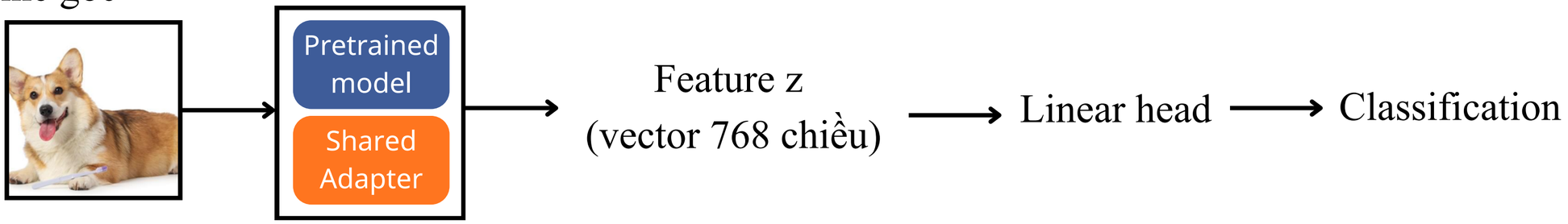
- Công thức:

$$\mathcal{L}_{\text{sep}}^{Q} = \frac{\sum_{(i,j)\in\mathcal{N}_{\text{cross}}} [K_{ij} - m_Q]_+}{|\mathcal{N}_{\text{cross}}| + \epsilon}$$

Đẩy xa khoảng cách giữa các mẫu của task mới với đại diện các lớp cũ, giữ cho biên giới phân chia giữa các task không bị chồng lấn.

Contribution ở giai đoạn suy luận

Baseline gốc



Quantum class representation

Biểu diễn các lớp đã học dưới dạng trạng thái lượng tử bằng cách sử dụng số liệu thống kê lớp đã lưu trữ, mà không cần giữ lại các mẫu huấn luyện.

Chuẩn hóa z, để lưu mean và covariance dựa trên base task : $v(z) = \begin{bmatrix} (z - m_0)/s_0 \\ 1 \end{bmatrix} \in \mathbb{R}^{769}$

Mã hóa biên độ (Amplitude Encoding), cho giai đoạn sau: $|\psi(z)\rangle = \frac{v(z)}{\|v(z)\|_2}$

Contribution ở giai đoạn suy luận

Quantum class representation

Chuẩn hóa z, để lưu mean và covariance dựa trên base task : $v(z) = \begin{bmatrix} (z - m_0)/s_0 \\ 1 \end{bmatrix} \in \mathbb{R}^{769}$

Vì adapter đã lưu được mean, covariance của từng class, đặt $u = \frac{z - m_0}{s_0}, v = \begin{bmatrix} u \\ 1 \end{bmatrix}$

$$vv^\top = \begin{bmatrix} uu^\top & u \\ u^\top & 1 \end{bmatrix} \xrightarrow{\text{Lấy kì vọng 2 vế}} \mathbb{E}[vv^\top] = \begin{bmatrix} \tilde{\Sigma}_c + \tilde{\mu}_c\tilde{\mu}_c^\top & \tilde{\mu}_c \\ \tilde{\mu}_c^\top & 1 \end{bmatrix} \xrightarrow{\text{Chuẩn hóa}} \rho_c = \frac{\mathbb{E}[vv^\top]}{\mathbb{E}[\|v\|_2^2]}$$

Mỗi ma trận là một biểu diễn của thông tin lớp tương ứng

$$\rho_{\text{Dog}}, \quad \rho_{\text{Cat}}, \quad \rho_{\text{Bird}}$$

Contribution ở giai đoạn suy luận

Construct measurement operators

Vấn đề mới là, vẫn chưa biết ảnh mới thuộc lớp nào ρ_{Dog}, ρ_{Cat}, ρ_{Bird}

$$\bar{\rho} = \frac{1}{C_t}\sum_{c=1}^{C_t}\rho_c \xrightarrow{\text{regularization}} \bar{\rho}_\delta = \bar{\rho} + \delta I \xrightarrow{\text{PGM}} E_c = \frac{1}{C_t}\bar{\rho}_\delta^{-1/2}\rho_c\bar{\rho}_\delta^{-1/2}$$

Thông tin đại diện cho lớp c

Điều chỉnh theo phân bố chung của các lớp

PGM không chỉ nhìn riêng từng lớp mà còn xét mối quan hệ giữa lớp đó với toàn bộ các lớp đã học.

Contribution ở giai đoạn suy luận

Compute measurement score

Biểu diễn ảnh đầu vào dưới dạng density matrix $\rho_z = |\psi(z)\rangle\langle\psi(z)|$

Điểm đo thô (raw measurement score) $q_c(z) = \mathrm{Tr}(E_c\rho_z) = \langle\psi(z)|E_c|\psi(z)\rangle$

Với một ảnh đầu vào, PGM tính điểm đo tương ứng với từng lớp đã học. Các điểm đo này cần được chuẩn hóa trên tất cả các lớp để tạo thành phân phối xác suất dự đoán.

$$p_{\mathrm{PGM}}(c|z) = \frac{\langle\psi(z)|E_c|\psi(z)\rangle}{\sum_{c'}\langle\psi(z)|E_{c'}|\psi(z)\rangle}$$

Class	PGM
Dog	0.75
Cat	0.15
Bird	0.1

Có thể dùng luôn PGM để suy luận, tuy nhiên lúc thực nghiệm kết quả không vượt đc 0.25 (A_B) nên là nhóm đưa ra thêm 1 đề xuất mới là kết hợp **linear head + PGM**

Contribution ở giai đoạn suy luận

Score-Calibrated Fusion (SCF)

Kết hợp thông tin từ PGM với Linear Head để tạo dự đoán cuối cùng. Do sự khác biệt về khoảng giá trị :

1. Linear Head tạo ra $\ell_c(z) \in (-\infty, +\infty)$ (logit chưa qua hàm softmax)
2. PGM tạo ra xác suất chuẩn hóa $p_{\text{PGM}}(c|z) \in [0, 1] \longrightarrow$ log-probability

Do giá trị của linear / log-PGM có thể bị chênh lệch nên cần 1 tham số học điều chỉnh, kappa điều chỉnh độ lớn score của PGM sao cho tương thích với độ lớn score của Linear Head.

$$\kappa = \frac{\text{sd}_c(\ell_c)}{\text{sd}_c(g_c)}$$

Nguồn dự đoán	Dog	Cat
Linear Head	2.0	2.5
PGM probability	0.7	0.3

Hàm mục tiêu: $\hat{y} = \arg\max_c \left[\ell_c(z) + w\kappa \log p_{\text{PGM}}(c|z)\right]$

Bản đối chứng cổ điển so với lượng tử

Sử dụng mean và covariance của các lớp trong Class Memory

$$W = \left[\sum_{c=1}^{C_t}(\Sigma_c + \mu_c\mu_c^\top) + \lambda I\right]^{-1}[\mu_1, \ldots, \mu_{C_t}]$$

Tính điểm dự đoán cho từng lớp $g_c^R(z) = z^\top W_{:,c}$

Hàm mục tiêu $\hat{y}_R = \arg\max_c \left[\ell_c(z) + w\kappa_R g_c^R(z)\right]$

Thí nghiệm thực nghiệm

- **Bài toán**: Exemplar-free CIL; đánh giá cumulative Top-1, không dùng Task ID.
- **Mô hình**: ViT-B/16 pretrained ImageNet-21K, backbone đóng băng, adapter dùng chung (RSIAT).
- **Lượng tử**: 8 qubit, 2 circuit layers, mô phỏng chính xác bằng statevector (tối đa 11 qubit).
- **Seed / class order**: 1993, 2015, 2026. Mọi lựa chọn siêu tham số chốt trên tuning group (ImageNet-R, seed 1993) rồi đóng băng.
- **Phần cứng**: NVIDIA RTX PRO 6000 Blackwell, PyTorch 2.11.

Kết quả thực nghiệm

Method	CIFAR-100		ImageNet-R		ImageNet-A		CUB-200		Average	
	$\bar{A}$	A_B	$\bar{A}$	A_B	$\bar{A}$	A_B	$\bar{A}$	A_B	$\bar{A}$	A_B
Reported by Zhao et al. [37] (exemplar-free, authors' partitions, one run)										
L2P [32]	89.23	84.29	75.46	69.77	49.39	41.71	67.05	56.25	70.28	63.01
DualPrompt [31]	87.36	82.30	73.10	67.18	53.71	41.67	77.47	66.54	72.91	64.42
CODA-Prompt [24]	91.30	86.85	77.97	72.27	53.54	42.73	84.00	73.37	76.70	68.80
APER+Adapter [41]	90.94	85.75	75.82	67.95	60.47	49.37	92.21	86.73	79.86	72.45
EASE [39]	92.35	87.76	81.74	76.17	65.34	55.04	92.23	86.81	82.91	76.45
SLCA [36]	94.09	91.53	81.17	77.00	68.66	58.74	89.51	82.19	83.36	77.36
RanPAC [19]	94.35	90.86	82.98	77.94	69.32	61.82	93.13	89.40	84.94	80.00
MOS [26]	94.75	91.70	82.96	77.93	69.13	59.12	93.49	90.12	85.08	79.72
SSIAT [27]	94.35	91.35	83.63	79.38	70.83	62.43	93.38	89.53	85.55	80.67
RSIAT [37]	95.15	92.20	86.92	82.75	74.89	66.23	93.99	90.59	87.74	82.94
Our runs (our partitions, mean ± SD over three seeds)										
RSIAT (ours)	95.02±.21	92.03±.19	87.11±.49	82.55±.05	74.26±1.1	67.02±1.1	93.53±.79	88.47±.31	87.48	82.52
SCF, ridge (classical)	95.22±.20	**92.41**±.21	87.42±.46	83.03±.24	**75.12**±1.2	**67.69**±0.9	**93.54**±.88	**88.85**±.22	**87.83**	**83.00**
SCF, PGM (quantum)	**95.23**±.22	92.34±.16	**87.50**±.43	**83.07**±.42	74.89±1.0	67.65±1.3	93.50±.84	88.57±.30	87.78	82.91
Δ ridge	+.20 [.12,.28]	+.38 [.08,.69]	+.32 [.12,.51]	+.48 [.02,.94]	+.85 [.62,1.08]	+.67 [−.20,1.54]	+.02 [−.24,.27]	+.38 [.16,.61]	+.35	+.48
Δ PGM	+.22 [.15,.28]	+.31 [.18,.43]	+.39 [.25,.53]	+.52 [−.41,1.45]	+.63 [−.10,1.36]	+.62 [−.08,1.32]	−.03 [−.18,.12]	+.10 [.05,.16]	+.30	+.39